

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В. И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №7
по дисциплине «Вычислительная математика»
Тема: Решение систем линейных уравнений методом Гаусса.

Студент гр. 4342

Алюкин Г.А.

Преподаватель

Пуеров Г.Ю.

Санкт-Петербург

2026

Цель работы.

Изучить и реализовать метод Гаусса для решения систем линейных алгебраических уравнений, включая выбор ведущего элемента, проверку совместности системы и контроль невырожденности матрицы в процессе вычислений; провести тестирование реализации на базовых, специальных (матрица Гильберта) и больших по размерности системах с анализом корректности, точности, времени и потребления памяти.

Теоретический минимум.

Метод Гаусса применяется для решения системы линейных уравнений вида $A * x = b$, где A - матрица коэффициентов размером $m \times n$, x - вектор неизвестных размерности n , b - вектор правых частей размерности m . Идея метода состоит в том, чтобы с помощью элементарных преобразований строк привести систему к эквивалентному более простому виду, из которого неизвестные находятся последовательно.

На прямом ходе метода для каждого столбца выбирается ведущий элемент (pivot), после чего элементы ниже него зануляются по правилу: $R_i := R_i - (a_{ik} / a_{kk}) * R_k$. В результате матрица приводится к верхнетреугольному (или ступенчатому) виду. Чтобы уменьшить численные ошибки, обычно используют выбор ведущего элемента по максимальному модулю в текущем столбце (частичный выбор ведущего элемента).

После прямого хода выполняется обратный ход. Неизвестные вычисляются снизу вверх по формуле: $x_i = (b_i - \text{сумма по } j \text{ от } i+1 \text{ до } n (a_{ij} * x_j)) / a_{ii}$. Это дает решение при наличии нужного числа ненулевых ведущих элементов.

Проверка существования и числа решений выполняется через ранги. Если $\text{rank}(A) = \text{rank}([A|b])$, система совместна. Если при этом $\text{rank}(A) = \text{rank}([A|b]) = n$, решение единственное. Если $\text{rank}(A) = \text{rank}([A|b]) < n$, система имеет бесконечно много решений. Если $\text{rank}(A) \neq \text{rank}([A|b])$, система несовместна и решений нет. В процессе исключения это видно по специальным строкам: строка вида $[0 \ 0 \ \dots \ 0 \mid c]$, где $c \neq 0$, означает несовместность; строка $[0 \ 0 \ \dots \ 0 \mid 0]$ означает линейную зависимость и наличие свободных переменных.

Для квадратной матрицы важна проверка невырожденности. Эквивалентные условия невырожденности: $\det(A) \neq 0$, $\text{rank}(A) = n$, а также наличие n ведущих элементов при прямом ходе. Если $\det(A) = 0$ (или $\text{rank}(A) < n$), матрица вырождена.

Из-за вычислений с плавающей точкой в численных методах используется порог eps : элементы, модуль которых меньше eps , считаются нулевыми. Для оценки качества найденного решения используют невязку $r = A * x - b$ и ее норму $\|A * x - b\|$. Если известно точное решение x^* , дополнительно оценивают ошибку $\|x - x^*\|$.

Для плотной квадратной системы размером $n \times n$ вычислительная сложность метода Гаусса составляет примерно $O(n^3)$ по времени и $O(n^2)$ по памяти. Поэтому для больших размерностей важны эффективная реализация, выбор ведущего элемента и контроль численной устойчивости.

Выполнение работы.

Описание структуры кода.

В ходе работы была реализована программа решения СЛАУ методом Гаусса на C++ с контролем корректности входа, выбором ведущего элемента, проверкой совместности/невырожденности в процессе вычислений, а также полноценным тестовым стендом с измерением времени и памяти.

1. Архитектура программы

Программа состоит из трёх логических уровней:

- Уровень данных — константа `eps`, перечисление `SolveStatus`, структура `GaussResult`.
- Уровень алгоритма — функция `gaussSolve()`, реализующая метод Гаусса.
- Уровень ввода/вывода — функция `main()`, отвечающая за чтение входных данных и печать результата.

2. Формат ввода и вывода

Ввод

Первая строка содержит два целых числа — количество уравнений m и количество неизвестных n .

Далее идут m строк по n вещественных чисел каждая (матрица A).

Последняя строка содержит m чисел - вектор правой части b .

m n

$A[0][0]$ $A[0][1]$... $A[0][n-1]$

...

$A[m-1][0]$... $A[m-1][n-1]$

$b[0]$ $b[1]$... $b[m-1]$

Вывод

Программа всегда выводит статус, ранг и признак невырожденности.

Вектор решения выводится только при совместной системе.

Статус: единственное / бесконечно много / нет решений / ошибка ввода

Ранг: Целое число - ранг матрицы A

Невырожденная: да / нет

Вектор x: n чисел через пробел (только если система совместна)

3. Ключевые типы и структуры

Константа точности

Используется единственная константа $\text{eps} = 1\text{e-}12$ для всех сравнений с нулём. Числа с плавающей точкой после арифметических операций никогда не бывают точно равны нулю, поэтому вместо проверки $== 0$ везде используется $\text{abs}(x) \leq \text{eps}$.

Перечисление SolveStatus

Вместо условных строк или «магических» чисел тип результата кодируется перечислением:

- Unique — единственное решение.
- Infinite — бесконечно много решений (есть свободные переменные).
- NoSolution — система несовместна.
- InputError — некорректный ввод.

Структура GaussResult

Функция-решатель возвращает один объект, содержащий все результаты:

- Status - Тип решения (SolveStatus)
- Rank - Ранг матрицы A
- Nonsingular - Признак невырожденности
- X - Вектор решения (частное при Infinite)

4. Реализация алгоритма Гаусса

Функция `gaussSolve()` реализует метод Гаусса с частичным выбором ведущего элемента в три фазы: прямой ход, проверка совместности, обратный ход.

1. Таблица ведущих элементов (`pivot_row`)

До начала прямого хода создаётся массив `pivot_row[n]`, заполненный значением -1 . После обработки столбца `col` в него записывается номер строки, в которой находится ведущий элемент: `pivot_row[col] = row`. Столбцы, пропущенные из-за нулевого максимума, остаются со значением -1 — это свободные переменные. Таблица используется в обратном ходе для восстановления соответствия «столбец $\rightarrow$ строка».

2. Прямой ход

Внешний цикл идёт по столбцам слева направо, а не по строкам — именно такая схема корректно работает с прямоугольными и вырожденными матрицами. Указатель `row` отстаёт от `col` всякий раз, когда столбец пропускается. Каждая итерация внешнего цикла состоит из трёх шагов.

Шаг 1. Выбор ведущего элемента (частичный поворот)

В необработанной части столбца `col` (строки `row..m-1`) ищется строка с наибольшим по модулю значением. Такой выбор называется частичным поворотом (`partial pivoting`).

При делении на ведущий элемент относительная погрешность обратно пропорциональна его величине. Если выбрать малый ведущий, ошибки округления резко возрастут. Выбор максимума минимизирует этот эффект.

Если найденный максимум $\leq \epsilon$ — весь столбец в необработанной части нулевой. Столбец `col` становится свободной переменной: `pivot_row[col]` остаётся -1 , указатель `row` не сдвигается, цикл переходит к следующему столбцу.

Шаг 2. Перестановка строк

Строка с максимальным элементом перемещается на позицию `row` с помощью `std::swap(A[row], A[max_row])`. Поскольку `std::vector` хранит данные по указателю, `swap` меняет только указатели — операция выполняется за $O(1)$ без копирования элементов.

Шаг 3. Обнуление строк ниже

Для каждой строки i ниже row вычисляется множитель:

$$factor = A[i][col] / A[row][col]$$

Затем из строки i вычитается ведущая строка, умноженная на $factor$:

$$A[i][j] -= factor * A[row][j] \quad (\text{для } j \text{ от } col+1 \text{ до } n-1)$$

$$b[i] -= factor * b[row]$$

После вычитания $A[i][col]$ теоретически должен стать нулём, но из-за ошибок округления там может оказаться значение порядка $1e-17$. Чтобы это не влияло на последующие итерации, выполняется явное присваивание $A[i][col] = 0.0$. Цикл по j начинается с $col+1$, так как левее col уже стоят нули из предыдущих итераций. По завершении шага ранг увеличивается на 1, указатель row сдвигается вперёд.

3. Проверка совместности системы

После прямого хода указатель row стоит на первой «лишней» строке. Все строки с индексами $row..m-1$ должны быть полностью нулевыми — как в матрице A , так и в векторе b .

Строка вида $[0 \ 0 \ \dots \ 0 \mid c \neq 0]$ означает уравнение $0 \cdot x_1 + 0 \cdot x_2 + \dots = c$, что является противоречием. При обнаружении такой строки функция немедленно возвращает статус `NoSolution`.

4. Обратный ход

Вектор x инициализируется нулями. Цикл идёт по столбцам справа налево. Для каждого столбца col проверяется таблица `pivot_row[col]`:

- Если `pivot_row[col] == -1` — столбец свободный, $x[col]$ остаётся 0 (частное решение при свободных переменных, равных нулю).
- Иначе — берётся строка $cur = pivot_row[col]$ и вычисляется неизвестное по формуле подстановки:

$$x[col] = (b[cur] - \sum A[cur][j] * x[j], j \text{ от } col+1 \text{ до } n-1) / A[cur][col]$$

5. Классификация результата

После обратного хода тип решения определяется по соотношению ранга и числа неизвестных

Условие	Статус	Невырожденная
$\text{rank} == n, m == n$	Unique (единственное)	да
$\text{rank} == n, m \neq n$	Unique (единственное)	нет (не квадратная)
$\text{rank} < n$	Infinite (бесконечно много)	нет
Противоречие $0=c \neq 0$	NoSolution (нет решений)	нет

6. Проверка невырожденности

Матрица называется невырожденной, если её определитель не равен нулю. Для квадратной матрицы $n \times n$ это эквивалентно тому, что ранг равен n . Признак невырожденности (поле nonsingular) выставляется в true только при одновременном выполнении двух условий: матрица квадратная ($m == n$) и $\text{rank} == n$. Для прямоугольных матриц понятие невырожденности не применяется.

7. Числовая погрешность и её контроль

Вещественная арифметика на компьютере приближённая: результат каждой операции может отличаться от математически точного на величину порядка машинного эпсилон ($\approx 2.2 \cdot 10^{-16}$ для double). В методе Гаусса ошибки накапливаются.

Программа контролирует погрешности способами:

- Порог сравнений: Единый порог $\text{eps} = 1\text{e-}12$.
- Частичный поворот: Выбор максимального по модулю элемента в качестве ведущего минимизирует деление на малые числа — главную причину роста ошибок.

8. Вычислительная сложность

Алгоритм работает непосредственно с исходными массивами. Асимптотическая сложность.

- Прямой ход (основная петля) - $O(m \cdot n \cdot \min(m, n))$
- Проверка совместности - $O(m \cdot n)$
- Обратный ход - $O(n^2)$
- Итого - $O(m \cdot n \cdot \min(m, n))$

Тестирование.

Код тестов приведен в Приложении А. Тестирование приведено в таблице 1 в Приложении Б.

Вывод.

Разработан и отлажен метод Гаусса на C++ с выбором главного элемента по столбцу. Алгоритм корректно распознает тип системы и факт вырожденности матрицы через анализ ранга. Тесты на матрице Гильберта и крупных размерностях (до 10^4) показали надежность кода и приемлемую точность вычислений. Выполнен анализ вычислительной сложности на практике для больших плотных систем.

ПРИЛОЖЕНИЕ А

Название файла: main.cpp

```
#include <iostream>
#include <vector>
#include <cmath>
#include <iomanip>

const double eps = 1e-12;

// Возможные типы решения системы
enum class SolveStatus { Unique, Infinite, NoSolution, InputError };

// Структура результата
struct GaussResult {
    SolveStatus status;
    int rank;           // ранг матрицы A
    bool nonsingular;   // true, если матрица квадратная и rank
== n
    std::vector<double> x; // вектор решения (частное, если Infinite)
};

// Метод Гаусса с частичным выбором ведущего элемента
//   A – матрица коэффициентов m×n (копия, изменяется внутри)
//   b – вектор правой части размера m (копия, изменяется внутри)
GaussResult gaussSolve(std::vector<std::vector<double>>> A,
std::vector<double> b, int m, int n)
{
    GaussResult res;
    res.rank = 0;
    res.nonsingular = false;

    // pivot_row[col] = r : в столбце col ведущий элемент стоит в
строке r.
    // -1 означает, что столбец пропущен (свободная переменная).
    std::vector<int> pivot_row(n, -1);

    int row = 0; // строка, куда кладём очередной ведущий элемент
```

```

// ПРЯМОЙ ХОД
// Проходим по столбцам слева направо; row отстаёт от col,
// если встречаются нулевые столбцы.
for (int col = 0; col < n && row < m; ++col) {

    // 1. Поиск максимума
    // В необработанной части столбца col (строки row..m-1) ищем
    // строку с наибольшим по модулю значением – частичный поворот.
    int max_row = row;
    double max_val = std::abs(A[row][col]);

    for (int i = row + 1; i < m; ++i) {
        double val = std::abs(A[i][col]);
        if (val > max_val) {
            max_val = val;
            max_row = i;
        }
    }

    // Если макс  $\leq$  eps – весь столбец в текущей части нулевой.
    // Столбец col становится свободной переменной, row не
сдвигаем.

    if (max_val <= eps) {
        continue;
    }

    // 2. Перестановка
    // Поднимаем строку с ведущим элементом на позицию row.
    if (max_row != row) {
        std::swap(A[row], A[max_row]);
        std::swap(b[row], b[max_row]);
    }

    // ведущий элемент col теперь в строке row.
    pivot_row[col] = row;
}

```

```

        // 3. Обнуление
        // Для каждой строки i НИЖЕ row вычисляем множитель и вычитаем
        строку row,

        // чтобы занулить элемент A[i][col].
        for (int i = row + 1; i < m; ++i) {
            if (std::abs(A[i][col]) <= eps) {
                A[i][col] = 0.0; // явный сброс накопленной погрешности
                continue;
            }

            double factor = A[i][col] / A[row][col];

            // Обновляем элементы столбцов col+1..n-1 (левее col нули)
            for (int j = col + 1; j < n; ++j) {
                A[i][j] -= factor * A[row][j];
            }
            b[i] -= factor * b[row];

            // зануляем, чтобы не накапливалась погрешность
            A[i][col] = 0.0;
        }

        // Ведущ элемент зафиксирован – переходим к следующей строке
        ++res.rank;
        ++row;
    }

    // ПРОВЕРКА НА НЕСОВМЕСТИМОСТЬ
    // После прямого хода строки row..m-1 должны быть нулевыми.
    // Строка вида [0 0 ... 0 | c≠0] – противоречие (0 = c), система
    несовместна
    for (int i = row; i < m; ++i) {
        bool row_is_zero = true;
        for (int j = 0; j < n; ++j) {
            if (std::abs(A[i][j]) > eps) { row_is_zero = false; break; }
        }
    }
}

```

```

        if (row_is_zero && std::abs(b[i]) > eps) {
            // Обнаружено противоречие: 0 = b[i]
            res.status      = SolveStatus::NoSolution;
            res.nonsingular = false;
            return res;
        }
    }

    // ОБРАТНЫЙ ХОД
    // Строим частное решение, свободные переменные полагаем равными
0
    // Идём по столбцам справа налево
    res.x.assign(n, 0.0);

    for (int col = n - 1; col >= 0; --col) {
        // Если в этом столбце нет ведущего элемента — x[col] = 0
(свободная переменная)
        if (pivot_row[col] == -1) {
            continue;
        }

        int cur = pivot_row[col]; // строка с ведущим элементом в
столбце col

        //  $x[col] = (b[cur] - \sum_{j=col+1}^{n-1} A[cur][j] * x[j]) /$ 
A[cur][col]
        double sum = b[cur];
        for (int j = col + 1; j < n; ++j) {
            sum -= A[cur][j] * res.x[j];
        }
        res.x[col] = sum / A[cur][col];
    }

    // ИТОГОВАЯ КЛАССИФИКАЦИЯ
    if (res.rank == n && m == n) {

```

```

        // Квадратная матрица с полным рангом – единственное решение,
матрица невырождена
        res.status      = SolveStatus::Unique;
        res.nonsingular = true;
    } else if (res.rank == n) {
        // Переопределённая совместная система ( $m > n$ ) – единственное
решение
        res.status      = SolveStatus::Unique;
        res.nonsingular = false; // не квадратная – невырожденность
неприменима
    } else {
        // rank < n – есть свободные переменные, решений бесконечно
много
        res.status      = SolveStatus::Infinite;
        res.nonsingular = false;
    }

    return res;
}

// ввод, вызов решателя, вывод
int main() {
    std::ios::sync_with_stdio(false);
    std::cin.tie(nullptr);

    int m, n;
    if (!(std::cin >> m >> n) || m <= 0 || n <= 0) {
        std::cout << "статус: ошибка ввода\n";
        return 1;
    }

    std::vector<std::vector<double>> A(m, std::vector<double>(n));
    for (int i = 0; i < m; ++i)
        for (int j = 0; j < n; ++j)
            if (!(std::cin >> A[i][j])) {
                std::cout << "статус: ошибка ввода\n"; return 1;
            }
}

```



```

std::vector<double> b(m);
for (int i = 0; i < m; ++i)
    if (!(std::cin >> b[i])) {
        std::cout << "статус: ошибка ввода\n"; return 1;
    }

GaussResult res = gaussSolve(A, b, m, n);

// Вывод статуса
switch (res.status) {
    case SolveStatus::Unique:
        std::cout << "статус: единственное\n"; break;
    case SolveStatus::Infinite:
        std::cout << "статус: бесконечно много\n"; break;
    case SolveStatus::NoSolution:
        std::cout << "статус: нет решений\n"; break;
    default:
        std::cout << "статус: ошибка ввода\n"; break;
}

std::cout << "ранг: " << res.rank << "\n";
std::cout << "невырожденная: " << (res.nonsingular ? "да" : "нет") << "\n";

if (res.status == SolveStatus::Unique || res.status == SolveStatus::Infinite) {
    std::cout << std::fixed << std::setprecision(6);
    std::cout << "вектор x:";
    for (double xi : res.x) std::cout << " " << xi;
    std::cout << "\n";

    if (res.status == SolveStatus::Infinite)
        std::cout << "(частное решение: свободные переменные = 0)\n";
}

```

```

        return 0;
    }

```

Название файла: test.cpp

```

// ИСКЛЮЧИВ main
// g++ -O2 -std=c++17 -DNO_MAIN gauss.cpp test.cpp -o test

#include <iostream>
#include <fstream>
#include <vector>
#include <cmath>
#include <chrono>
#include <string>
#include <iomanip>
#include <sstream>
#include <functional>

#include "gauss.h"

struct TestResult {
    std::string name;           // название теста
    bool passed;               // OK / FAIL
    std::string fail_reason;    // причина провала (пусто при OK)
    double elapsed_ms;         // время работы, мс
    size_t memory_bytes;       // оценка расхода памяти, байт
    int rank;                  // ранг, полученный решателем
    bool nonsingular;          // признак невырожденности
    SolveStatus status;        // статус решения
};

// Глобальный лог: консоль + файл одновременно
struct DualLog {
    std::ofstream file;
    DualLog(const std::string& fname) : file(fname) {}

    template<typename T>
    DualLog& operator<<(const T& v) {
        std::cout << v;
        file << v;
        return *this;
    }
} log_("test_results.txt");

// Замер времени — возвращает миллисекунды
template<typename Fn>
double measure_ms(Fn&& fn) {
    auto t0 = std::chrono::high_resolution_clock::now();
    fn();
    auto t1 = std::chrono::high_resolution_clock::now();
    return std::chrono::duration<double, std::milli>(t1 - t0).count();
}

```

```

// Оценка памяти для матрицы m*n + вектора m (в байтах, без накладных
расходов)
size_t estimate_memory(int m, int n) {
    // матрица A: m*n double + m указателей vector
    // вектор b: m double
    // pivot_row: n int
    // вектор x: n double
    return static_cast<size_t>(m) * n * sizeof(double)
        + static_cast<size_t>(m) * sizeof(std::vector<double>) //
заголовки строк
        + static_cast<size_t>(m) * sizeof(double) // b
        + static_cast<size_t>(n) * sizeof(int) //
pivot_row
        + static_cast<size_t>(n) * sizeof(double); // x
}

// Невязка: max |A*x - b|
double residual(const std::vector<std::vector<double>>& A,
                const std::vector<double>& b,
                const std::vector<double>& x)
{
    int m = static_cast<int>(A.size());
    int n = static_cast<int>(x.size());
    double max_err = 0.0;
    for (int i = 0; i < m; ++i) {
        double s = 0.0;
        for (int j = 0; j < n; ++j) s += A[i][j] * x[j];
        max_err = std::max(max_err, std::abs(s - b[i]));
    }
    return max_err;
}

// Проверка x против известного эталона
double solution_error(const std::vector<double>& got,
                      const std::vector<double>& expected)
{
    double err = 0.0;
    for (size_t i = 0; i < got.size(); ++i)
        err = std::max(err, std::abs(got[i] - expected[i]));
    return err;
}

// Печать одной строки итогового отчёта
void print_result(const TestResult& r) {
    // Заголовок печатается один раз перед циклом, здесь только строка
результата
    log_ << (r.passed ? "[ OK ]" : "[FAIL]") << " "
    << std::left << std::setw(42) << r.name
    << std::right << std::setw(10) << std::fixed <<
std::setprecision(2) << r.elapsed_ms << " ms"
    << " | mem " << std::setw(10) << r.memory_bytes
    << " | rank " << std::setw(3) << r.rank
    << " | nonsing " << (r.nonsingular ? "Y" : "N");
    log_ << "\n";
}

```

```

// ТЕСТЫ

// Вспомогательная обёртка: запускает тест, замеряет время и память
TestResult run_test(
    const std::string& name,
    std::vector<std::vector<double>> A,
    std::vector<double> b,
    // принимает результат и возвращает "" (OK) или описание ошибки
    std::function<std::string(const GaussResult&,
                             const std::vector<std::vector<double>>&,
                             const std::vector<double>&)> check)
{
    TestResult tr;
    tr.name = name;
    tr.memory_bytes = estimate_memory(static_cast<int>(A.size()),
                                      static_cast<int>(A[0].size()));

    GaussResult res;
    tr.elapsed_ms = measure_ms([&]{ res = gaussSolve(A, b,
                                                      static_cast<int>(A.size()),
                                                      static_cast<int>(A[0].size()));
});
    tr.rank = res.rank;
    tr.nonsingular = res.nonsingular;
    tr.status = res.status;
    tr.fail_reason = check(res, A, b);
    tr.passed = tr.fail_reason.empty();
    return tr;
}

// Тест 1. Единственное решение 2x2
// 2x + y = 5
// x + 3y = 10 → x=1, y=3

TestResult test_unique_2x2() {
    std::vector<std::vector<double>> A = {{2, 1}, {1, 3}};
    std::vector<double> b = {5, 10};
    std::vector<double> expected = {1.0, 3.0};

    return run_test("Unique 2x2 (x=1, y=3)", A, b,
        [&](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Unique) return "ожидался
Unique";
            if (r.rank != 2) return "ожидался
rank=2";
            if (!r.nonsingular) return "ожидалась
невыврожденность";
            double err = solution_error(r.x, expected);
            if (err > 1e-9) return "погрешность решения " +
std::to_string(err);
            return "";
        });
}

// Тест 2. Единственное решение 3x3 (классический пример)

```

```
// 2x + y - z = 8
// -3x - y + 2z = -11
// -2x + y + 2z = -3 → x=2, y=3, z=-1
//
```

```
TestResult test_unique_3x3() {
    std::vector<std::vector<double>> A = {
        { 2, 1, -1},
        {-3, -1, 2},
        {-2, 1, 2}
    };
    std::vector<double> b = {8, -11, -3};
    std::vector<double> expected = {2.0, 3.0, -1.0};

    return run_test("Unique 3x3 (x=2, y=3, z=-1)", A, b,
        [&](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Unique) return "ожидался
Unique";
            if (r.rank != 3) return "ожидался
rank=3";
            if (!r.nonsingular) return "ожидалась
невыврожденность";
            double err = solution_error(r.x, expected);
            if (err > 1e-9) return "погрешность решения " +
std::to_string(err);
            return "";
        });
}
```

```
// Тест 3. Нет решений
// x + y = 1
// x + y = 2 → противоречие
```

```
TestResult test_no_solution() {
    std::vector<std::vector<double>> A = {{1, 1}, {1, 1}};
    std::vector<double> b = {1, 2};

    return run_test("NoSolution (противоречие)", A, b,
        [] (const GaussResult& r, auto&, auto&) -> std::string {
            if (r.status != SolveStatus::NoSolution) return "ожидался
NoSolution";
            if (r.rank != 1) return "ожидался
rank=1";
            return "";
        });
}
```

```
// Тест 4. Нет решений (3 уравнения, явное противоречие в третьем)
// x + 2y + 3z = 6
// 2x + 4y + 6z = 12
// x + 2y + 3z = 7 → последнее уравнение противоречит первому
```

```
TestResult test_no_solution_3x3() {
    std::vector<std::vector<double>> A = {
        {1, 2, 3},
```

```

        {2, 4, 6},
        {1, 2, 3}
    };
    std::vector<double> b = {6, 12, 7};

    return run_test("NoSolution 3x3 (строка 0≠c)", A, b,
        [](const GaussResult& r, auto&, auto&) -> std::string {
            if (r.status != SolveStatus::NoSolution) return "ожидался
NoSolution";
            return "";
        });
}

// Тест 5. Бесконечно много решений (недоопределённая система)
//   x + 2y + 3z = 6
//   2x + 4y + 6z = 12   →   rank=1, z и y свободны
//
//   Проверяем, что частное решение (свободные = 0) удовлетворяет
//   системе.

TestResult test_infinite() {
    std::vector<std::vector<double>> A = {
        {1, 2, 3},
        {2, 4, 6}
    };
    std::vector<double> b = {6, 12};

    return run_test("Infinite 2x3 (rank=1)", A, b,
        [](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Infinite) return "ожидался
Infinite";
            if (r.rank != 1) return "ожидался
rank=1";
            // Частное решение должно удовлетворять системе
            double res = residual(A2, b2, r.x);
            if (res > 1e-9) return "невязка частного решения " +
std::to_string(res);
            return "";
        });
}

// Тест 6. Бесконечно много решений – ранг 2, матрица 3×4
TestResult test_infinite_3x4() {
    // Строим систему: первые два уравнения независимы, третье = сумма
    // первых двух
    std::vector<std::vector<double>> A = {
        {1, 0, 1, 2},
        {0, 1, 2, 1},
        {1, 1, 3, 3} // = строка0 + строка1
    };
    std::vector<double> b = {3, 4, 7}; // b[2] = b[0]+b[1]

    return run_test("Infinite 3x4 (rank=2)", A, b,
        [](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Infinite) return "ожидался
Infinite";

```

```

        if (r.rank != 2)                                return "ожидался
rank=2";
        double res = residual(A2, b2, r.x);
        if (res > 1e-9) return "невязка частного решения " +
std::to_string(res);
        return "";
    });
}

// Тест 7. Матрица Гильберта 5×5 (плохо обусловленная)
//  H[i][j] = 1 / (i + j + 1)
//  Строим b = H * x_exact, где x_exact = (1,1,1,1,1), затем решаем.
//  Принимаем, если невязка < 1e-6 (число обусловленности ~5e5).
TestResult test_hilbert_5x5() {
    const int N = 5;
    std::vector<std::vector<double>> H(N, std::vector<double>(N));
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            H[i][j] = 1.0 / (i + j + 1);

    std::vector<double> x_exact(N, 1.0);
    std::vector<double> b(N, 0.0);
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            b[i] += H[i][j] * x_exact[j];

    return run_test("Hilbert 5x5 (точность, cond~5e5)", H, b,
        [&](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Unique) return "ожидался
Unique";
            if (!r.nonsingular)                        return "ожидалась
невыврожденность";
            double err = solution_error(r.x, x_exact);
            // Матрица Гильберта 5×5 – число обусловленности ~5e5,
            // допускаем погрешность до 1e-6
            if (err > 1e-6) return "погрешность " + std::to_string(err) +
" > 1e-6";
            return "";
        });
}

// Тест 8. Матрица Гильберта 8×8 (сильно плохо обусловленная)
//  Число обусловленности ~1.5e10 – проверяем лишь совместность и
//  невязку.
TestResult test_hilbert_8x8() {
    const int N = 8;
    std::vector<std::vector<double>> H(N, std::vector<double>(N));
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            H[i][j] = 1.0 / (i + j + 1);

    std::vector<double> x_exact(N, 1.0);
    std::vector<double> b(N, 0.0);
    for (int i = 0; i < N; ++i)
        for (int j = 0; j < N; ++j)
            b[i] += H[i][j] * x_exact[j];

```

```

return run_test("Hilbert 8x8 (сильный cond~1.5e10)", H, b,
    [&](const GaussResult& r, auto& A2, auto& b2) -> std::string {
        if (r.status != SolveStatus::Unique) return "ожидался
Unique";
        // При таком числе обусловленности допускаем невязку до 1e-4
        double res = residual(A2, b2, r.x);
        if (res > 1e-4) return "невязка " + std::to_string(res) + " >
1e-4";
        return "";
    });
}

// Тест 9. Диагональная матрица 1000x1000
// A = diag(1, 2, 3, ..., 1000)
// b[i] = (i+1) * (i+1) → x[i] = i+1
// Проверяем время, правильность, невязку.
TestResult test_diagonal_1000x1000() {
    const int N = 1000;
    std::vector<std::vector<double>> A(N, std::vector<double>(N, 0.0));
    std::vector<double> b(N), x_exact(N);
    for (int i = 0; i < N; ++i) {
        A[i][i] = static_cast<double>(i + 1);
        x_exact[i] = static_cast<double>(i + 1);
        b[i] = A[i][i] * x_exact[i]; // (i+1)^2
    }

    return run_test("Diagonal 1000x1000 (скорость)", A, b,
        [&](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Unique) return "ожидался
Unique";
            if (r.rank != N) return "ожидался rank=1000";
            if (!r.nonsingular) return "ожидалась невырожденность";
            double err = solution_error(r.x, x_exact);
            if (err > 1e-6) return "погрешность " + std::to_string(err);
            return "";
        });
}

// Тест 10. Нулевая матрица с ненулевым b (крайний случай)
// Все A[i][j] = 0, b = (1, 0) → нет решений
TestResult test_zero_matrix() {
    std::vector<std::vector<double>> A = {{0, 0}, {0, 0}};
    std::vector<double> b = {1, 0};

    return run_test("Zero matrix, b≠0 (NoSolution)", A, b,
        [&](const GaussResult& r, auto&, auto&) -> std::string {
            if (r.status != SolveStatus::NoSolution) return "ожидался
NoSolution";
            if (r.rank != 0) return "ожидался
rank=0";
            return "";
        });
}

// Тест 11. Нулевая матрица с нулевым b (бесконечно много решений)

```



```

// Все A[i][j] = 0, b = (0, 0) → бесконечно много
TestResult test_zero_matrix_zero_b() {
    std::vector<std::vector<double>> A = {{0, 0}, {0, 0}};
    std::vector<double> b = {0, 0};

    return run_test("Zero matrix, b=0 (Infinite)", A, b,
        [](const GaussResult& r, auto&, auto&) -> std::string {
            if (r.status != SolveStatus::Infinite) return "ожидался
Infinite";
            if (r.rank != 0) return "ожидался
rank=0";
            return "";
        });
}

// Тест 12. Переопределённая совместная система (3 уравнения, 2
неизвестных)
// x + y = 3
// 2x - y = 0 → x=1, y=2
// x - y = -1
TestResult test_overdetermined() {
    std::vector<std::vector<double>> A = {
        {1, 1},
        {2, -1},
        {1, -1}
    };
    std::vector<double> b = {3, 0, -1};
    std::vector<double> expected = {1.0, 2.0};

    return run_test("Overdetermined 3x2, Unique (x=1,y=2)", A, b,
        [&](const GaussResult& r, auto& A2, auto& b2) -> std::string {
            if (r.status != SolveStatus::Unique) return "ожидался
Unique";
            if (r.rank != 2) return "ожидался
rank=2";
            double err = solution_error(r.x, expected);
            if (err > 1e-9) return "погрешность " + std::to_string(err);
            return "";
        });
}

// main: запуск всех тестов и печать итогового отчёта
int main() {
    // Собираем все тесты в список
    std::vector<std::function<TestResult()>> tests = {
        test_unique_2x2,
        test_unique_3x3,
        test_no_solution,
        test_no_solution_3x3,
        test_infinite,
        test_infinite_3x4,
        test_hilbert_5x5,
        test_hilbert_8x8,
        test_diagonal_1000x1000,
        test_zero_matrix,
        test_zero_matrix_zero_b,
    };
}

```

```

        test_overdetermined,
    };

    log_ << " Тесты метода Гаусса с частичным выбором ведущего
элемента\n";

    std::vector<TestResult> results;
    results.reserve(tests.size());

    for (auto& t : tests) {
        TestResult r = t();
        results.push_back(r);
        print_result(r);
    }

    // Итоговая статистика
    int ok    = 0;
    int fail = 0;
    for (auto& r : results) r.passed ? ++ok : ++fail;

    log_ << "\nИтого: " << ok << " ОК,  " << fail << " FAIL"
        << "   (из " << results.size() << " тестов)\n";

    if (fail > 0) {
        log_ << "\nПровалившиеся тесты:\n";
        for (auto& r : results)
            if (!r.passed)
                log_ << "   * " << r.name << " → " << r.fail_reason <<
"\n";
    }

    log_ << "Сохранено в test_results.txt\n";

    return fail == 0 ? 0 : 1;
}

```

ПРИЛОЖЕНИЕ В

ТЕСТИРОВАНИЕ

Результаты тестирования

№	Ввод (с описанием)	Вывод (включая решение)	Время	Память
1	Базовый уникальный случай: $A = \begin{bmatrix} 2 & 1 \\ 1 & 3 \end{bmatrix}$, $b = [5, 10]$ (квадратная невырожденная 2×2)	Единственное решение, $\text{rank} = 2$, невырожденность: да, $x = [1.000000, 3.000000]$	0.001 мс	120.00 Б
2	Квадратная 3×3 : $A = \begin{bmatrix} 2 & 1 & -1 \\ -3 & -1 & 2 \\ -2 & 1 & 2 \end{bmatrix}$, $b = [8, -11, -3]$ (невырожденная, требует перестановки строк)	Единственное решение, $\text{rank} = 3$, невырожденность: да, $x = [2.000000, 3.000000, -1.000000]$	0.005 мс	204 Б
3	Несовместная 3×3 : строка 3 = строка 1, но $b[2] \neq b[0]$ (скрытое противоречие $0 = 1$)	Решений нет, $\text{rank} = 1$, невырожденность: нет, x отсутствует	0.001 мс	204 Б
4	Недоопределённая 3×4 : $\text{rank} = 2$, третье уравнение = сумма первых двух	Бесконечно много решений, $\text{rank} = 2$, невырожденность: нет, частное $x = [3.000000, 4.000000, 0.000000, 0.000000]$	0.002 мс	240 Б
5	Матрица Гильберта 5×5 : $H[i][j] = 1/(i+j+1)$, $x_{\text{exact}} = [1, 1, 1, 1, 1]$ ($\text{cond} \approx 5 \cdot 10^5$)	Единственное решение, $\text{rank} = 5$, невырожденность: да, погрешность $< 1e-6$	0.001 мс	420 Б

6	Матрица Гильберта 8×8 : $H[i][j] = 1/(i+j+1)$, $x_{\text{exact}} = [1, \dots, 1]$ ($\text{cond} \approx 1.5 \cdot 10^{10}$)	Единственное решение, $\text{rank} = 8$, невыврожденность: да, невязка $< 1e-4$	0.002 мс	864 Б
7	Диагональная 1000×1000 : $A[i][i] = i+1$, $b[i] = (i+1)^2$, $x_{\text{exact}}[i] = i+1$ (проверка масштаба)	Единственное решение, $\text{rank} = 1000$, невыврожденность: да, погрешность $< 1e-6$	7.122 мс	7.86 МБ
8	Нулевая матрица, $b \neq 0$: $A = [[0,0],[0,0]]$, $b = [1,0]$ (крайний случай)	Решений нет, $\text{rank} = 0$, невыврожденность: нет, x отсутствует	0.001 мс	120 Б
9	Переопределённая 3×2 : 3 уравнения, 2 неизвестных, все совместны, $x_{\text{exact}} = [1,2]$	Единственное решение, $\text{rank} = 2$, невыврожденность: нет (не квадратная), $x = [1.000000, 2.000000]$	0.002 мс	168 Б

Таблица 1 – Тестирование алгоритма.

Примеры запуска

Единственное решение

3 3

2 1 -1

-3 -1 2

-2 1 2

8 -11 -3

Нет решений

2 2

1 1

1 1

1 2

Бесконечно много (недоопределённая)

1 3
1 2 3
6